

AIDA vs Continue

AIDA positioning · best-effort comparison — live source:
github.com/joemooney/aida

rendered 2026-07-10

Last updated: 2026-07-09

The TL;DR: **different layers that compose.** Continue is an AI coding assistant (CLI + VS Code / JetBrains) that, since its 2026 pivot, leans **CI-native** — declarative, source-controlled checks that run against your changes. AIDA is the requirement graph + life-cycle above the editing. Continue is excellent at “*did this change pass our rules?*”; AIDA answers “*what was this change supposed to implement, and is that spec now done?*” They sit at different layers and the interesting move is running them together.

Like Aider and Cline, Continue is an **adjacent neighbor, not a direct competitor** — it optimizes for a different job.

What Continue is

Continue is an open-source AI code assistant for the CLI, VS Code, and JetBrains. Its distinctive 2026 direction is **CI-native, declarative checks**: each check is a **markdown file** in `.continue/checks/`, source-controlled and reviewable, that the assistant runs against your work.

Be precise about its strengths:

- **Declarative, reviewable checks.** Putting quality gates in version-controlled mark-down — *the check is a file* — is a clean pattern. It makes “what does good look like here?” a reviewable artifact, not tribal knowledge.
- **Editor-native.** It lives where you already write code (VS Code / JetBrains), not in a separate surface.
- **Open core, commercializing.** Continue moved OSS → commercial in 2026; the editor/checks core remains open, but verify current licensing and hosting before assuming no lock-in.

What Continue is *not*

- It has **no cross-feature spec graph** — `.continue/checks/` are quality rules, not a typed graph of requirements with stable IDs and relationships.

- It doesn't model a **lifecycle** — there's no Draft → Approved → Done → Completed state machine, no queue, no auto-bump on merge.
- Its checks describe *standards*, not *intent* — “code must have tests” is a rule; “this function implements STORY-4's tag-filter” is a trace. Different artifact.

Where Continue holds up (and you should just use it)

- You want editor-native AI assistance with reviewable, source-controlled quality gates.
- Your need is “enforce our standards on every change,” not “track a graph of intent across the project's life.”
- You're not coordinating multiple agents/sessions around a durable spec graph.

If that's the shape, **use Continue**. AIDA's lifecycle machinery wouldn't add value over what Continue already gives you.

Where AIDA adds a layer Continue doesn't have

When the question shifts from “*does this change meet our bar?*” to “*what spec does this change implement, what else is under that epic, what's blocked, and is the spec now Completed?*” — that's the requirement-graph-and-lifecycle layer Continue doesn't model. AIDA's stable IDs, typed relationships, code↔spec traces, and merge-driven status bumps are that layer.

How they compose

Run Continue as the **editing + standards-check layer** and AIDA as the **spec-graph + lifecycle layer** above it:

- **AIDA** files the spec, routes it to a queue, records the // trace: linkage, and bumps status on merge.
- **Continue** edits in your IDE and runs its .continue/checks/ gates on the change before it lands.
- Together: the change is both *standards-checked* (Continue) and *tied to its intent and lifecycle* (AIDA).

Worth borrowing: Continue's markdown-as-CI is a clean pattern for AIDA's own reviewer phase — declarative check files instead of `claude -p` invocations is a tracked opportunity. Good ideas travel between neighbors.

Bottom line

Continue enforces *how* code should look and behave on its way in. AIDA remembers *what* the code was for and *whether the spec is done*. Use Continue for editor-native, reviewable quality gates; reach for AIDA when you need the durable graph of intent — and layer them when you want both.

See also

- [composition.md](#) — the general “use AIDA *with* an editor” recipe.
- [vs-aider.md](#) — the auto-commit-per-turn editor neighbor.
- [when-not-to-use-aida.md](#) — when Continue alone is enough.